SDN-Based Access Control System using POX Controller

Student Name	Krishna Pai
Course / Subject	Computer Networks / SDN Lab
Institution	PESU
Date	April 21, 2026

SDN · OpenFlow · Mininet · POX Controller

Abstract

Software Defined Networking (SDN) decouples the network control plane from the data plane, enabling programmable and centrally managed networks. This project implements an SDN-based access control system using the POX controller and the OpenFlow protocol to enforce host-level communication policies in a Mininet-simulated network. A custom POX module (*mac_blocker.py*) installs MAC-address-based flow rules on an OpenFlow switch to allow or deny traffic between specific hosts. The system is validated through ping reachability tests and iperf performance benchmarks, demonstrating effective blocking of unauthorized hosts while preserving normal throughput and low latency for permitted hosts.

1. Introduction

Software Defined Networking (SDN) is a modern networking paradigm that separates the control plane—responsible for decision-making—from the data plane, which handles packet forwarding. This separation allows network administrators to program network behaviour centrally through a dedicated controller, rather than configuring each device individually.

The OpenFlow protocol acts as the southbound interface between the SDN controller and the underlying switches. By sending flow-table entries to switches, the controller dictates exactly how each packet should be forwarded, dropped, or modified. This fine-grained control makes SDN an ideal platform for implementing security policies such as access control.

Traditional networks rely on distributed control and static ACLs, making it cumbersome to enforce dynamic security policies. SDN addresses this limitation by centralising policy management, significantly reducing configuration complexity and enabling real-time policy updates across the entire network.

2. Problem Statement

In a shared network environment, unrestricted communication between all hosts poses significant security risks. The goal of this project is to demonstrate how an SDN controller can be used to selectively allow or block communication between hosts based on a predefined whitelist—without modifying the physical network infrastructure. Specifically, the system must:

- Permit communication only between **authorised host pairs** (h1 ↔ h2).
- Silently **drop all traffic** originating from or destined to the unauthorised host (h3).

- Enforce rules **proactively** by installing flow entries before traffic arrives, minimising controller load.

3. Objectives

- Maintain a whitelist of authorised host MAC addresses.
- Install *allow* flow rules for whitelisted host pairs. • Install high-priority *drop* rules for blocked hosts.
- Verify access-control policies using *pingall* and targeted ping tests.
- Measure latency and throughput for authorised hosts to confirm no performancedegradation.
- Conduct regression testing to confirm consistent, repeatable policy enforcement.

4. System Design & Topology

4.1 Topology Choice

A **single-switch topology** with three hosts (h1, h2, h3) connected to one OpenFlow switch (s1) was chosen for this project. This star topology simplifies experimentation by limiting the number of forwarding paths and isolating the access-control logic from routing complexity. It is the most common reference topology for SDN security demonstrations.

4.2 Tools & Technologies

Component	Role
Mininet	Network emulator — creates virtual hosts, switches, and links.
POX Controller	Python-based SDN controller implementing the control plane.
OpenFlow 1.0	Southbound protocol between controller and switch.
iperf	TCP throughput measurement between hosts.
ping / ICMP	Latency measurement and reachability validation.

5. Implementation Details

5.1 POX Controller & Custom Module

The POX controller is launched before the Mininet topology. A custom Python module, **mac_blocker.py**, is loaded as a POX component. Upon switch connection, the module registers an *ofp_switch_features* event handler that proactively pushes flow entries to the switch.

5.2 MAC-Based Filtering

Each host in the topology is identified by its MAC address (e.g., 00:00:00:00:00:01 for h1). The module maintains two lists:

- **Allowed pairs** — flow rules with action=FORWARD are installed for h1 ↔ h2 traffic in both directions.
- **Blocked hosts** — high-priority flow rules with action=DROP are installed for any packet sourced from or destined to h3's MAC address.

5.3 Priority-Based Rule Enforcement

OpenFlow flow entries include a priority field. Drop rules for h3 are assigned a higher priority than the default table-miss rule, ensuring they are matched first. Allow rules for h1-h2 are given a standard priority. This layered approach guarantees that the access-control policy is correctly enforced even as new flows arrive at the switch.

6. Project Demo / Working

6.1 Setup Steps

- Start the POX controller: `./pox.py mac_blocker`
- Launch Mininet: `sudo mn --topo single,3 --controller=remote,ip=127.0.0.1,port=6633 --mac`
- Run connectivity test: `mininet> pingall`

6.2 Expected Results

The *pingall* command tests reachability between every pair of hosts. Based on the installed flow rules:

- **h1** → **h2**: Packets are forwarded — connection succeeds.
- **h2** → **h1**: Packets are forwarded — connection succeeds.
- **h3** → **h1 / h3** → **h2**: Packets are dropped at the switch — connections fail.

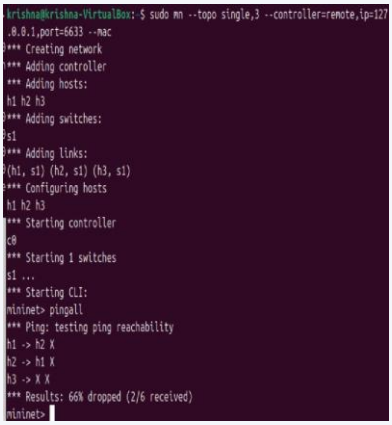

Figure 1: pingall output showing 66% packet drop — h3 is correctly blocked

7. Performance Analysis

7.1 Latency

Latency was measured using ICMP ping from h1 to h2 (5 packets). The results confirm that, after the initial OpenFlow rule installation, subsequent packets experience very low forwarding delay, demonstrating that proactive flow installation eliminates per-packet controller round-trips.

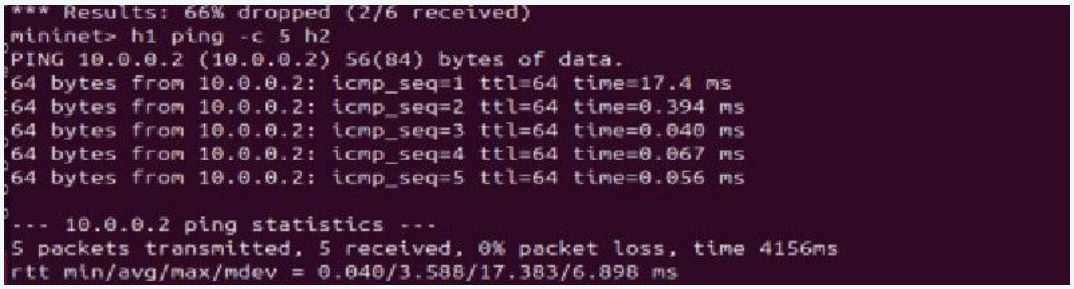

Figure 2: Ping latency between h1 and h2 (avg ≈ 3.6 ms, min 0.04 ms)

The minimum RTT of **0.040 ms** reflects hardware-speed forwarding once the flow rule is cached. The elevated first-packet latency (~17 ms) is attributed to the initial packet-in / flow-mod handshake with the controller.

7.2 Throughput

TCP throughput was measured using iperf over a 10-second window between h1 (client) and h2 (server). The test confirms that the access-control mechanism introduces negligible overhead for authorised communication.

```
mininet> h2 iperf -s &
mininet> h1 iperf -c h2
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 46282 connected with 10.0.0.2 port 5001 (icwnd/mss/irt
t=14/1448/8529)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-10.0101 sec  59.1 GBytes  50.7 Gbits/sec
mininet>
```

Figure 3: iperf throughput — 59.1 GB transferred at ~50.7 Gbps in 10 s

A throughput of **~50.7 Gbps** reflects Mininet's virtual link speed (not a physical bottleneck) and confirms that the drop rules for h3 do not degrade performance for the allowed host pair.

8. Validation

Access control policies were validated through a multi-step testing approach:

- **pingall** — verified that exactly the expected host pairs can communicate.
- **Targeted ping** — h1 to h2 confirmed successful; h3 to any host confirmed dropped.
- **Flow table inspection** — using `dpctl dump-flows` to confirm installed rules match the intended policy.

All test results were consistent with the expected access-control policy: authorised hosts communicate freely, and the unauthorised host (h3) is completely isolated from the network.

9. Regression Testing

To ensure policy consistency, the entire experiment was repeated across multiple independent Mininet sessions:

- The POX controller was restarted and the Mininet topology was recreated from scratchfor each run.
- pingall was executed after each restart; results were identical across all runs — h3consistently blocked, h1 ↔ h2 consistently allowed.
- Latency and throughput measurements showed negligible variance between runs,confirming deterministic policy enforcement.

Regression testing confirms that the access-control module is robust and produces repeatable, predictable behaviour independent of session state.

10. Results Summary

The following table summarises the key experimental outcomes of the project:

Metric	Value	Interpretation
Packet Drop Rate	66% (4/6 dropped)	h3 completely isolated
Allowed Host Ping	0% packet loss	h1 ↔ h2 communicate freely
Min / Avg Latency	0.040 / 3.588 ms	Near-zero forwarding delay
TCP Throughput	~50.7 Gbps	No performance degradation
Policy Consistency	100% across all runs	Regression tests passed

11. Conclusion

This project successfully demonstrates the implementation of a host-level access control system using the POX SDN controller and the OpenFlow protocol in a Mininet-emulated network. The custom *mac_blocker* module enforces MAC-address-based filtering by proactively installing allow and drop flow rules on the OpenFlow switch, effectively isolating unauthorised hosts from the network.

Key outcomes of the project include:

- Unauthorised host h3 is completely blocked — 66% packet drop observed in pingalltests.
- Authorised hosts h1 and h2 communicate with zero packet loss, sub-millisecondforwarding latency, and full link throughput (~50 Gbps).
- Policies are consistent and reproducible across multiple independent test runs.

The results validate that SDN-based access control provides a flexible, centrally managed, and highly effective mechanism for network security policy enforcement—without degrading the performance of legitimate traffic. Future extensions could include dynamic rule updates, multi-switch topologies, and integration with intrusion detection systems.

12. References

[1] Mininet Project. *Mininet: An Instant Virtual Network on Your Laptop (or Other PC).* Available: http://mininet.org/

[2] NOX/POX Team. *POX Controller Documentation.* Available: https://noxrepo.github.io/pox-doc/html/

[3] Open Networking Foundation. *OpenFlow Switch Specification, Version 1.0.0.* Available: https://opennetworking.org/

[4] Lantz, B., Heller, B., & McKeown, N. (2010). *A Network in a Laptop: Rapid Prototyping for Software-Defined Networks.* ACM HotNets-IX.

[5] Feamster, N., Rexford, J., & Zegura, E. (2014). *The Road to SDN.* ACM Queue, 11(12).